

RIX 2024- Batch 1_ Software & Manual Testing Practical Assessment

12576 Nisanth Saravanan

Online Courier Management System

An online courier management system is a software application that is used to manage the process of sending and tracking packages and documents through a courier service. This typically includes functions such as creating shipping labels, tracking packages, and generating reports. This project is a useful tool for businesses and individuals who need to send packages and documents through a courier service on a regular basis.

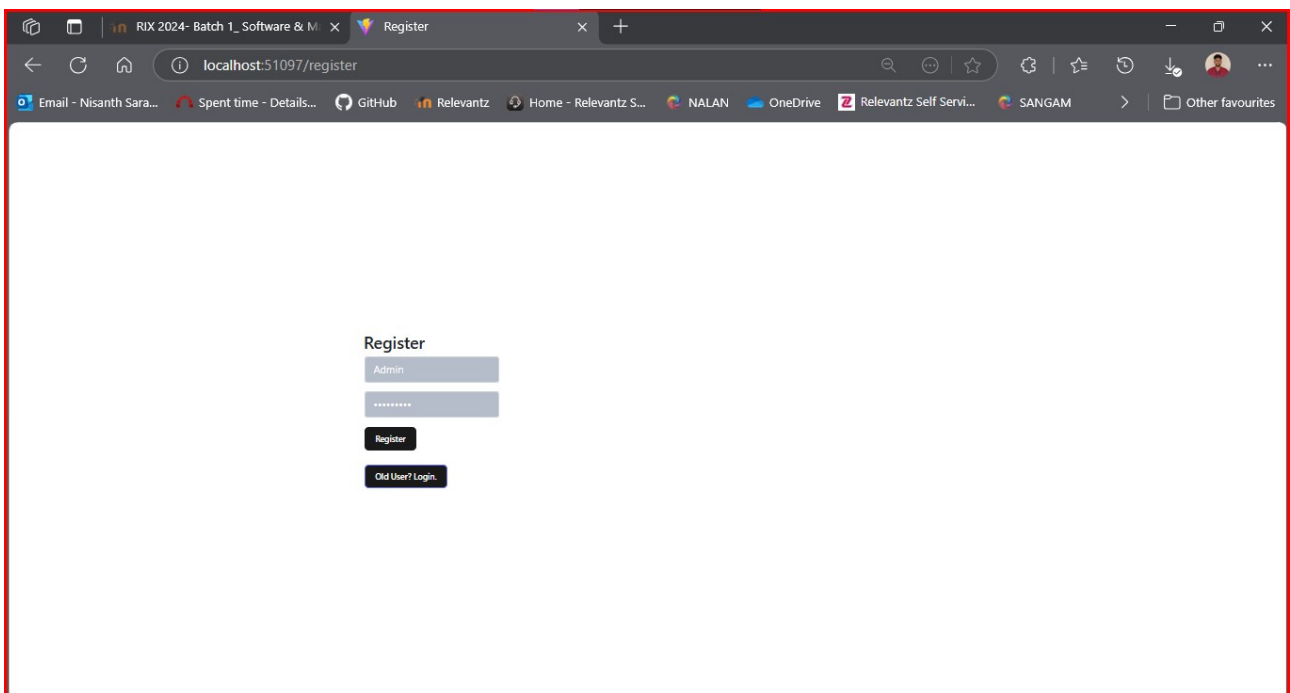
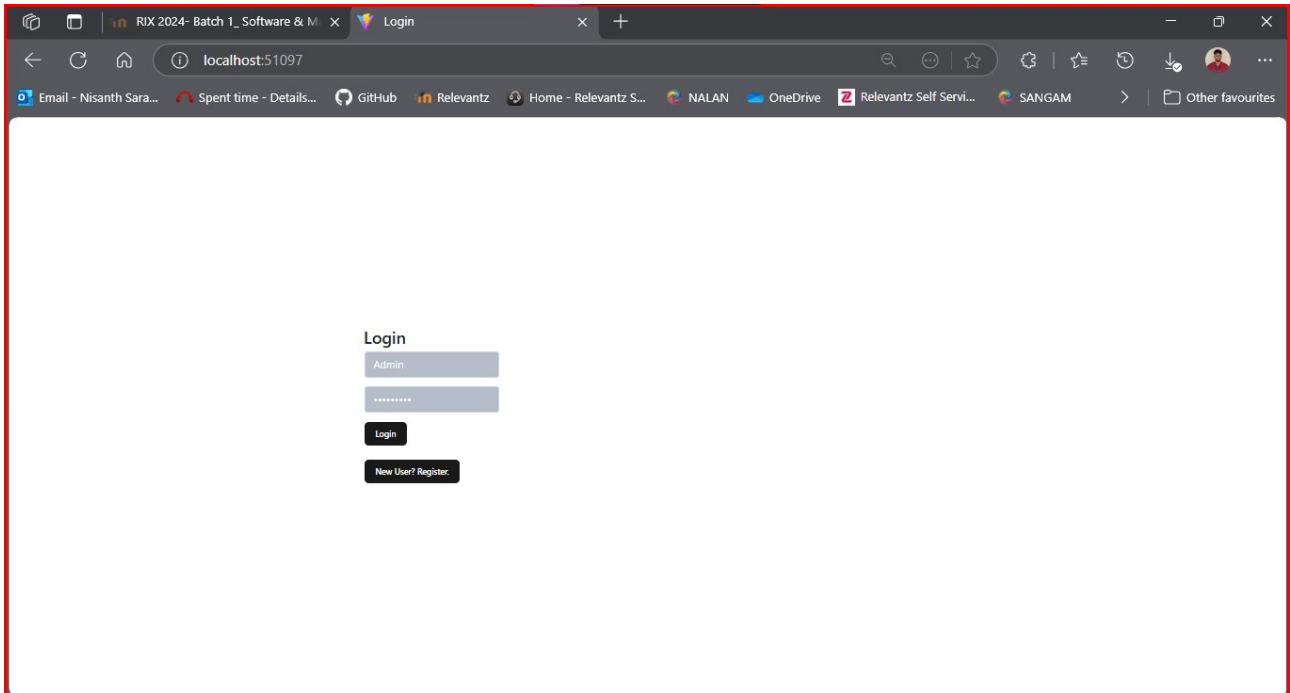
W3H Analysis

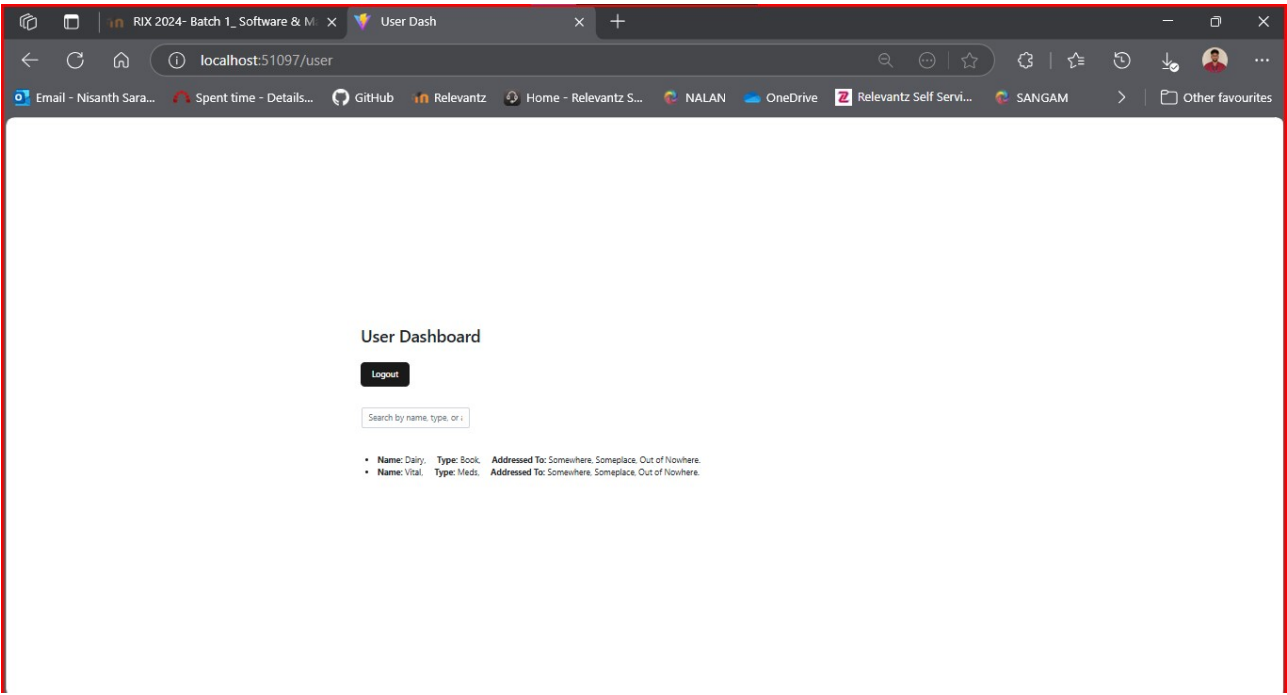
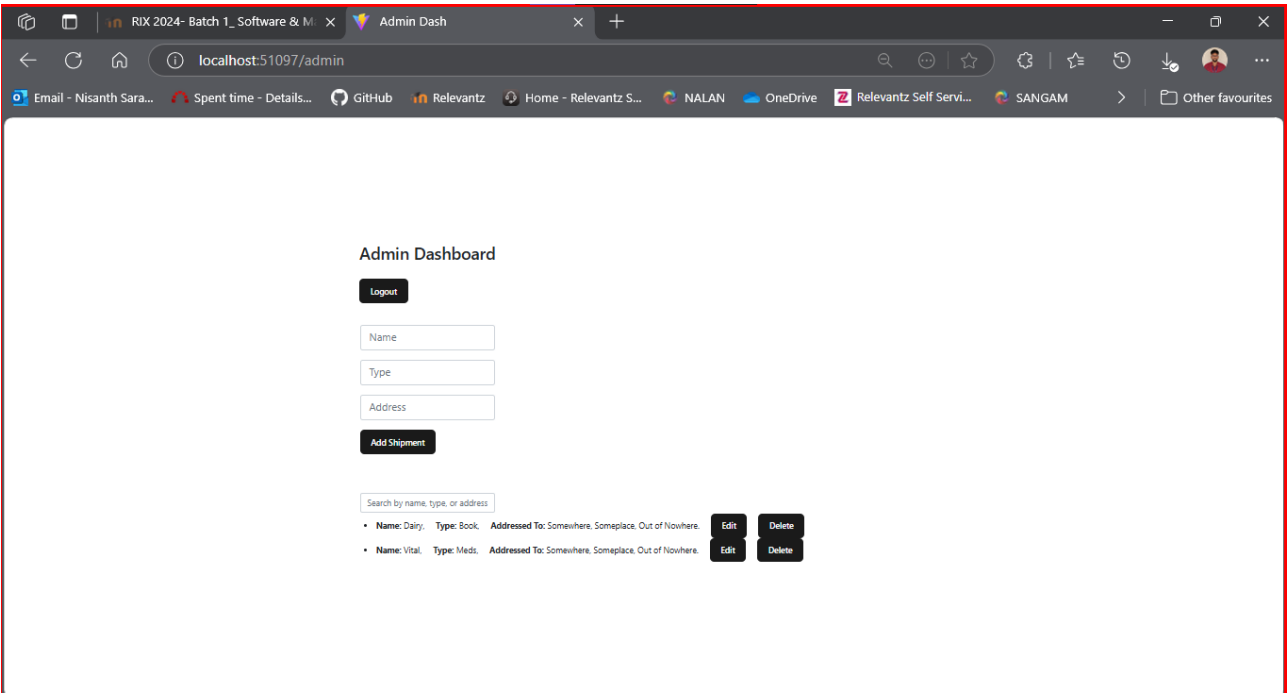
Project Title: Online Courier Management System	
What?	How?
1. What are the necessary modules? A: Modules Needed: 1) Login and Registration. 2) Admin Dashboard. 3) User Dashboard. 2. What are the necessary functionalities of Login and Registration Modules? A: Login and Registration Module: 1) Register User. 2) Login as User. 3) Login as Admin. 3. What are the necessary functionalities of Admin Dashboard? A: Admin Dashboard: 1) Perform CRUD Operation with Shipments. 2) Filter Shipments. 4. What are the necessary functionalities of User Dashboard? A: User Dashboard: 1) Filter Shipments.	Login and Registration Module: 1) Register User: Method 1: Use Simple Form Page. Method 2: Use a Modular Form Page. Method 3: Use Google OAuth and other 3rd party services. 2) Login as User: Method 1: Use Simple Form. Method 2: Use a Modular Form. Method 3: Use 3rd party services. 3) Login as Admin: Method 1: Hard Code the Admin accounts. Method 2: Have an Admin Login Page. Method 3: Have a separate Register + Login for admins. Admin Dashboard: 1) Perform CRUD Operation with Shipments: Method 1: Use Simple Forms and Buttons to perform CRUD. Method 2: Have separate pages for each operation Method 3: Use Modals. 2) Filter Shipments: Method 1: Use a simple search bar. Method 2: Have tables and sorting buttons. Method 3: Have a separate page to pick out exact results. User Dashboard: 1) Filter Shipments: Method 1: Use a simple search bar.

	Method 2: Have tables and sorting buttons. Method 3: Have a separate page to pick out exact results.
Login and Registration Module: 1) Register User: Method 1: Use Simple Form Page. A: Fits the use case as a simple and easy to work with solution. It is also effective in resolving the requirement in hand. 2) Login as User: Method 1: Use Simple Form. A: Fits the use case as a simple and easy to work with solution. It is also effective in resolving the requirement in hand. 3) Login as Admin: Method 1: Hard Code the Admin accounts. A: Fits the use case as a simple and easy to work with solution. It is also effective in resolving the requirement in hand. Admin Dashboard: 1) Perform CRUD Operation with Shipments: Method 1: Use Simple Forms and Buttons to perform CRUD. A: Fits the use case as a simple and easy to work with solution. It is also effective in resolving the requirement in hand. 2) Filter Shipments: Method 1: Use a simple search bar. A: Fits the use case as a simple and easy to work with solution. It is also effective in resolving the requirement in hand. User Dashboard: 1) Filter Shipments: Method 1: Use a simple search bar. A: Fits the use case as a simple and easy to work with solution. It is also effective in resolving the requirement in hand.	Login and Registration Module: 1) Register User: Method 2: Use a Modular Form Page. Method 3: Use Google OAuth and other 3rd party services. A: Increases the complexity of the application with no valid reason in hand. Also increases the overall scale of the project with no time to support its completion. 2) Login as User: Method 2: Use a Modular Form. Method 3: Use 3rd party services. A: Increases the complexity of the application with no valid reason in hand. Also increases the overall scale of the project with no time to support its completion. 3) Login as Admin: Method 2: Have an Admin Login Page. Method 3: Have a separate Register + Login for admins. A: Increases the complexity of the application with no valid reason in hand. Also increases the overall scale of the project with no time to support its completion. Admin Dashboard: 1) Perform CRUD Operation with Shipments: Method 2: Have separate pages for each operation Method 3: Use Modals. A: Increases the complexity of the application with no valid reason in hand. Also increases the overall scale of the project with no time to support its completion. 2) Filter Shipments: Method 2: Have tables and sorting buttons. Method 3: Have a separate page to pick out exact results.

	A: Increases the complexity of the application with no valid reason in hand. Also increases the overall scale of the project with no time to support its completion. User Dashboard: 1) Filter Shipments: Method 2: Have tables and sorting buttons. Method 3: Have a separate page to pick out exact results. A: Increases the complexity of the application with no valid reason in hand. Also increases the overall scale of the project with no time to support its completion.
Why?	Why Not?

Output Screenshots





Testing and Result:

Full File:



12576_Nisanth_Saravanan_MannualTesting.xlsx

Login Module:

Project name :	Online Courier Management System
Module Name :	Login Module
Reference Document :	
Created by :	Nisanth Saravanan
Date of creation :	05-08-2025
Date of review :	

Login_Pass	User enters the credentials and proceeds to login.	Login Success	Enter User Name.	No User is signed in.	An alert box pops up with login success message.	An alert box pops up with login success message.	Pass	
			Enter Password.	Valid Username	User is redirected to the page according to their role.	User is redirected to the page according to their role.		
			Click on the "Login" Button.	Valid Password				

Login

Login

New User? Register.

Admin Dashboard

Logout

Add Shipment

Login

User Dashboard

- **Name:** Dairy, **Type:** Book, **Addressed To:** Somewhere, Someplace, Out of Nowhere.
- **Name:** Vital, **Type:** Meds, **Addressed To:** Somewhere, Someplace, Out of Nowhere.

Login_Fail1	User enters the credentials and proceeds to login.	Invalid Username	Enter User Name.	No User is signed in.	An alert box pops up with login failure message.	An alert box pops up with login failure message.	Pass	
			Enter Password.	Invalid Username	Login Page is re-rendered	Login Page is re-rendered		
			Click on the "Login" Button.	Valid Password				

Login

wsretyuijoi

...

Login

New User? Register.

localhost:51097 says

Invalid credentials

OK

Login_Fail2	User enters the credentials and proceeds to login.	Invalid Password	Enter User Name.	No User is signed in.	An alert box pops up with login failure message.	An alert box pops up with login failure message.	Pass	
			Enter Password.	Valid Username	Login Page is re-rendered	Login Page is re-rendered		
			Click on the "Login" Button.	Invalid Password				

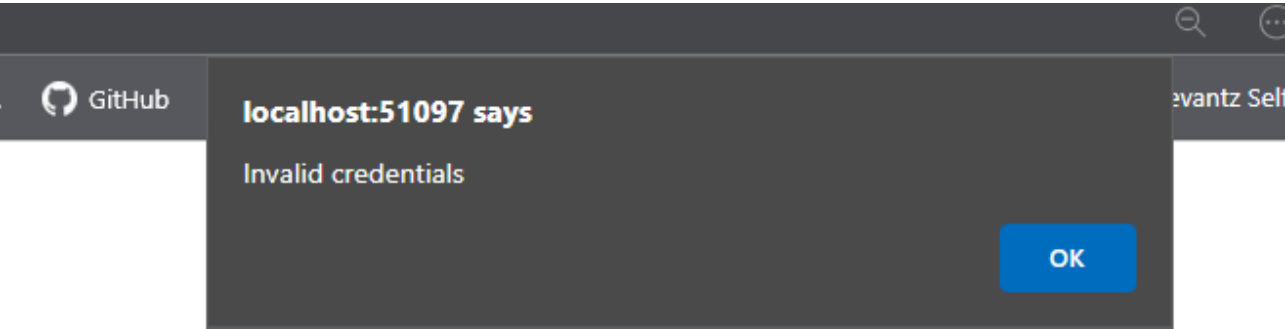
Login

Admin

.....

Login

New User? Register.



Login

Admin

.....

Login

New User? Register.

Login_Fail3	User enters the credentials and proceeds to login.	Invalid Credentials	Enter User Name.	No User is signed in.	An alert box pops up with login failure message.	An alert box pops up with login failure message.	Pass	
			Enter Password.	Invalid Username	Login Page is re-rendered	Login Page is re-rendered		
			Click on the "Login" Button.	Invalid Password				

Login

setrdyft

.....

Login

New User? Register.

GitHub

localhost:51097 says

Invalid credentials

OK

evantz

Login

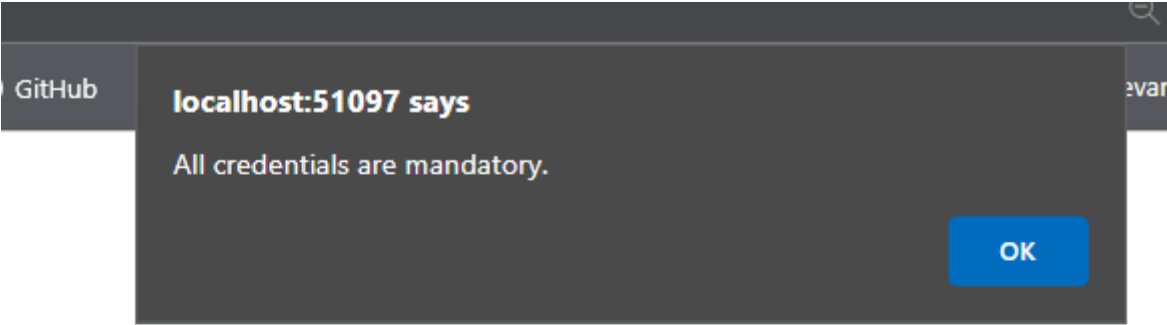
setrdyft

.....

Login

New User? Register.

NoData	User enters the credentials and proceeds to login.	No data	Click on the "Login" Button.	No User is signed in.	An alert box pops up with login failure message.	An alert box pops up with login failure message.	Pass	
				No data in input.	Login Page is re-rendered	Login Page is re-rendered		



Login

To_Register	User wishes to redirect to register page to register.	Successful redirection.	Click on the "New User? Register" Button	No User is signed in.	Page is redirected and "Register" Page is rendered.	Page is redirected and "Register" Page is rendered.	Pass	
				No data in input.				

Login

Username

Password

Login

New User? Register.

Register

User

...

Register

Old User? Login.

Register Module

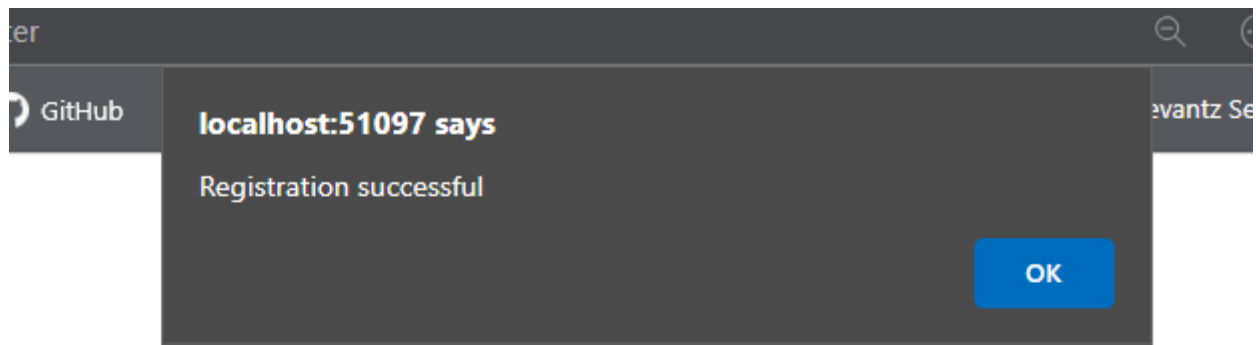
Project name :	Online Courier Management System
Module Name :	Register Module
Reference Document :	
Created by :	Nisanth Saravanan
Date of creation :	05-08-2025
Date of review :	

Test Case ID	Test Scenarios	Test Case Title / Test Case Name	Test Steps	Precondition/Test Data	Expected Result	Actual Result	Status	Comments
Register_Pass	User enters the credentials and proceeds to Register.	Register Success	Enter User Name.	No User is signed in.	An alert box pops up with Register success message.	An alert box pops up with Register success message.	Pass	
			Enter Password.	Valid Username				
			Click on the "Register" Button.	Valid Password				

Register

Register

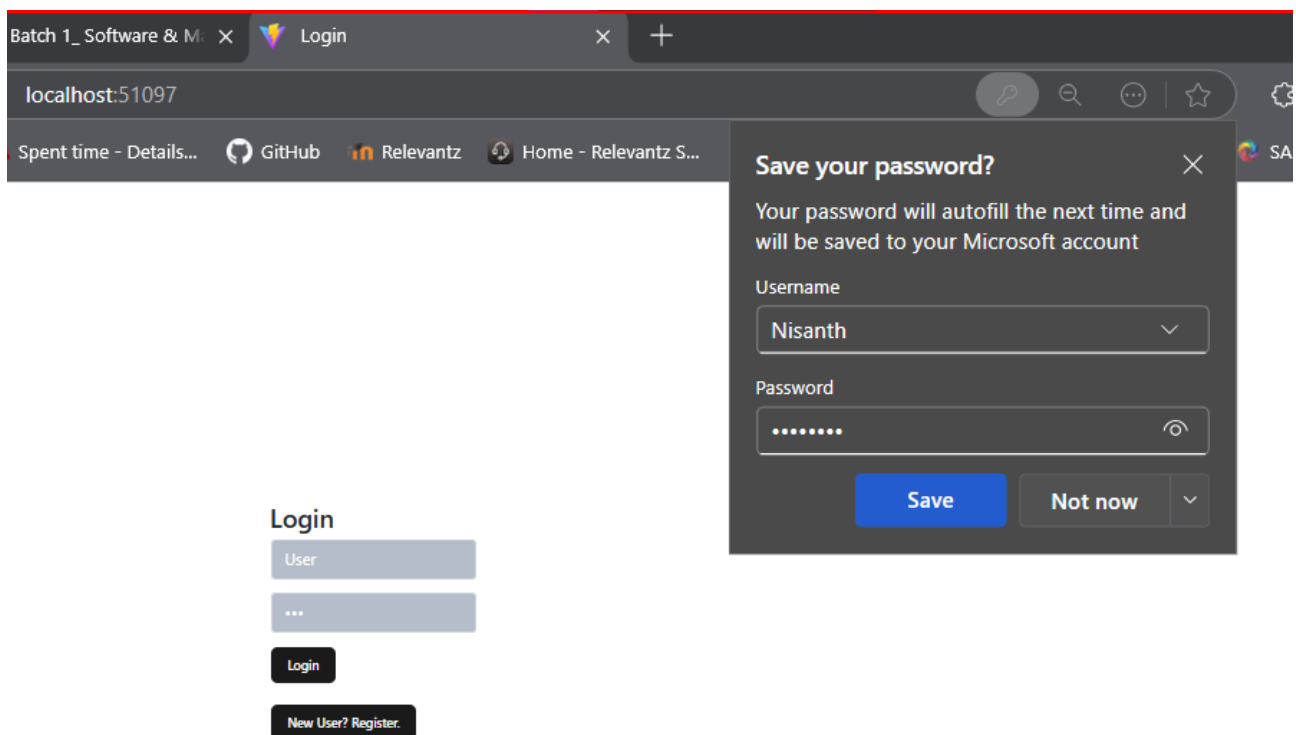
Old User? Login.



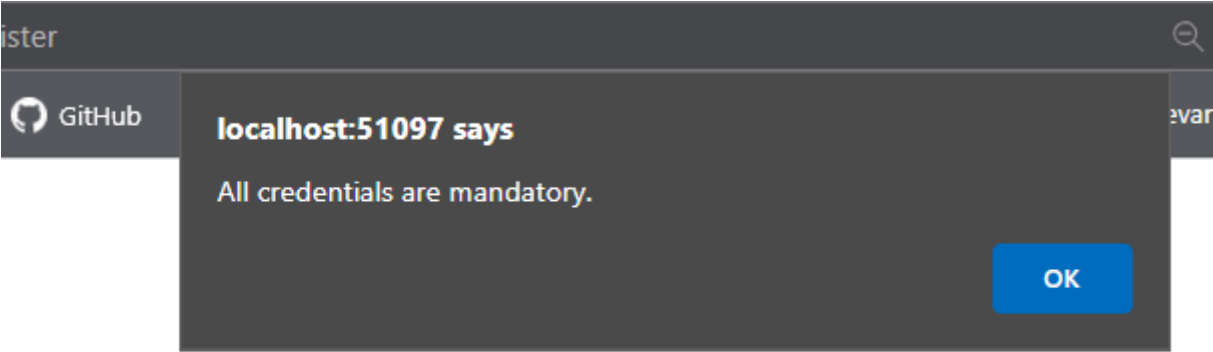
Register

Register

Old User? Login.



NoDate	User enters the credentials and proceeds to Register.	No data	Click on the "Register" Button.	No User is signed in.	An alert box pops up with Register failure message.	An alert box pops up with Register failure message.	Pass	
				No data in input.	Register Page is re-rendered	Register Page is re-rendered		



Register

Register

Old User? Login.

To_Login	User wishes to redirect to Login page to Login.	Successful redirection.	Click on the "Old User? Login" Button	No User is signed in.	Page is redirected and "Login" Page is rendered.	Page is redirected and "Login" Page is rendered.	Pass	
				No data in input.				

Register

Username

Password

Register

Old User? Login.

Login

Nisanth

.....

Login

New User? Register.

User Module

Project name :	Online Courier Management System
Module Name :	User Module
Reference Document :	
Created by :	Nisanth Saravanan
Date of creation :	05-08-2025
Date of review :	

Test Case ID	Test Scenarios	Test Case Title / Test Case Name	Test Steps	Precondition/Test Data	Expected Result	Actual Result	Status	Comments
Logout_Success	User wishes to log out.	Successful log out.	Click on the "Logout" Button.	Logged in as "User".	User is logged out.	User is logged out.	Pass	
					Redirected to Login Page.	Redirected to Login Page.		

User Dashboard

Logout

Login

Login

New User? Register.

User_Search	User wishes to search for a particular shipment in specific.	Search Success.	Type with the name or the type or the address of a shipment in the search bar.	Logged in as "User".	The List of shipment is filtered dynamically to show the results.	The List of shipment is filtered dynamically to show the results.	Pass	
				Data is in the search bar.				

User Dashboard

Logout

Search by name, type, or i

- **Name:** Dairy, **Type:** Book, **Addressed To:** Somewhere, Someplace, Out of Nowhere.
- **Name:** Vital, **Type:** Meds, **Addressed To:** Somewhere, Someplace, Out of Nowhere.

User Dashboard

Logout

Dai

- **Name:** Dairy, **Type:** Book, **Addressed To:** Somewhere, Someplace, Out of Nowhere.

User Dashboard

Logout

V

- **Name:** Vital, **Type:** Meds, **Addressed To:** Somewhere, Someplace, Out of Nowhere.

User Dashboard

Logout

Boo

- **Name:** Dairy, **Type:** Book, **Addressed To:** Somewhere, Someplace, Out of Nowhere.

User Dashboard

Logout

Me

- **Name:** Vital, **Type:** Meds, **Addressed To:** Somewhere, Someplace, Out of Nowhere.

Admin Module

Project name :	Online Courier Management System
Module Name :	Admin Module
Reference Document :	
Created by :	Nisanth Saravanan
Date of creation :	05 08 2025
Date of review :	

Test Case ID	Test Scenarios	Test Case Title / Test Case Name	Test Steps	Precondition/Test Data	Expected Result	Acutal Result	Status	Comments
Logout_Success	Admin wishes to logout.	Successful log out.	Click on the "Logout" Button.	Logged in as "Admin".	Admin is logged out.	Admin is logged out.	Pass	
					Redirected to Login Page.	Redirected to Login Page.		

Admin Dashboard

Logout

Login

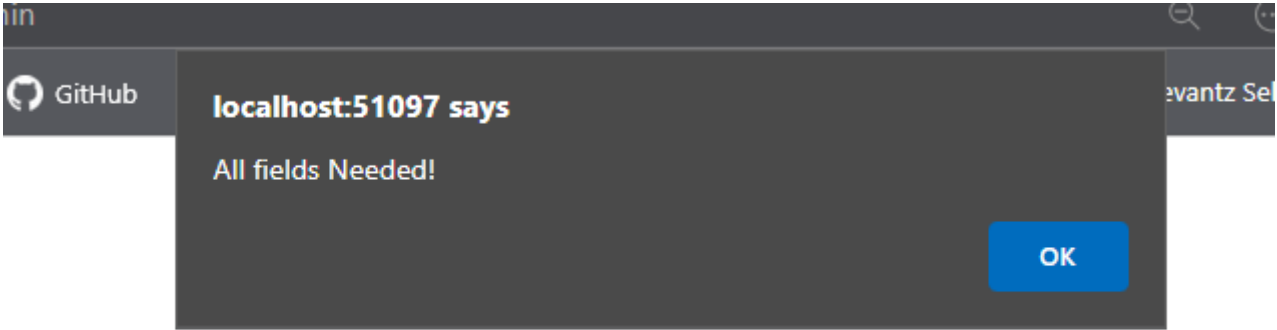
Admin

.....

Login

New User? Register.

Admin_Add_Shipment2	Admin wished to add a new shipment.	Create Failure	Click on the "Add" button.	Logged in as "Admin".	An alter is given stating that all fields are mandatory.	An alter is given stating that all fields are mandatory.	Pass	
				NO fields are entered.				



Admin Dashboard

Logout

Name

Type

Address

Add Shipment

Admin_Add_Shipment1	Admin wished to add a new shipment.	Create Success	Enter "Name"	Logged in as "Admin".	Shipment is added and the page is re-rendered.	Shipment is added and the page is re-rendered.	Pass	
			Enter "Type"	All fields are entered.	The creation is reflected in the database.	The creation is reflected in the database.		
			Enter "Address"					
			Click on the "Add" button.					

Admin Dashboard

Logout

Cards

Toys

Nothern Uganda

Add Shipment

Search by name, type, or address

- **Name:** Dairy, **Type:** Book, **Addressed To:** Somewhere, Someplace, Out of Nowhere.
- **Name:** Vital, **Type:** Meds, **Addressed To:** Somewhere, Someplace, Out of Nowhere.

Edit

Delete

Edit

Delete

Search by name, type, or address

- **Name:** Dairy, **Type:** Book, **Addressed To:** Somewhere, Someplace, Out of Nowhere.
- **Name:** Vital, **Type:** Meds, **Addressed To:** Somewhere, Someplace, Out of Nowhere.
- **Name:** Cards, **Type:** Toys, **Addressed To:** Nothern Uganda.

Edit

Delete

Edit

Delete

Edit

Delete

Admin_Add_Shipment3	Admin wished to add a new shipment.	Back-end not running	Enter "Name"	Logged in as "Admin".	Failure status is given out.	Failure status is given out.	Pass	
			Enter "Type"	All fields are entered.	No reflections being made.	No reflections being made.		
			Enter "Address"		Console provides internal server error.	Console provides internal server error.		
			Click on the "Add" button.					

Admin Dashboard

Logout

Vital 2345

Meds

Somewhere, Someplace, (

Add Shipment

```
SELECT `s` . id , `s` . Address , `s` .  
FROM `Shipments` AS `s`  
info: Microsoft.Hosting.Lifetime[0]  
Application is shutting down...
```

▶ Uncaught (in promise) ▶ AxiosError {message: 'Network Error', name: 'AxiosError', code: 'ERR_NETWORK', config: {...}, request: XMLHttpRequest, ...}	AdminDashboard.jsx:34
▶ POST http://localhost:5099/api/shipment?role=Admin net::ERR_CONNECTION_REFUSED	AdminDashboard.jsx:30

Admin_Edit	Admin wishes to edit a shipment	Edit Success	Click on the "Edit" Button	Logged in as "Admin".	Edit is successful.	Edit is successful.	Pass	
			Edit the values in the form.	Some fields are Edited.	The page is re-rendered.	The page is re-rendered.		
			Click on the "Update Shipment" Button		The reflection is made in database.	The reflection is made in database.		

Search by name, type, or address

• **Name:** Dairy, **Type:** Book, **Addressed To:** Somewhere, Someplace, Out of Nowhere.

Edit

Delete

Admin Dashboard

Logout

Dairy

Book and Persona

Somewhere, Someplace, Out of Nowhere

Update Shipment

Search by name, type, or address

• **Name:** Dairy, **Type:** Book and Personal, **Addressed To:** Somewhere, Someplace, Out of Nowhere.

Edit

Delete

Admin_Delete	Admin wishes to Delete a shipment	Delete Success	Click on the "Delete" Button in a shipment.	Logged in as "Admin".	Deletion is successful.	Deletion is successful.	Pass
					The page is re-rendered.	The page is re-rendered.	
					The reflection is made in database.	The reflection is made in database.	

Search by name, type, or address

- Name: Dairy, Type: Book and Personal, Addressed To: Somewhere, Someplace, Out of Nowhere.

EditDelete
- Name: Vital, Type: Meds, Addressed To: Somewhere, Someplace, Out of Nowhere.

EditDelete
- Name: Cards, Type: Toys, Addressed To: Nothern Uganda.

EditDelete

Search by name, type, or address

- Name: Vital, Type: Meds, Addressed To: Somewhere, Someplace, Out of Nowhere.

EditDelete
- Name: Cards, Type: Toys, Addressed To: Nothern Uganda.

EditDelete

Admin_Search	Admin wishes to search for a particular shipment in specific.	Search Success.	Type wither the name or the type or the address of a shipment in the search bar.	Logged in as "Admin".	The List of shipment is filtered dynamically to show the results.	The List of shipment is filtered dynamically to show the results.	Pass	
				Data is in the search bar.				

Admin Dashboard

Logout

Name

Type

Address

Add Shipment

Vi

- Name: Vital, Type: Meds, Addressed To: Somewhere, Someplace, Out of Nowhere.

Edit

Delete

ca

- Name: Cards, Type: Toys, Addressed To: Nothern Uganda.

Edit

Delete

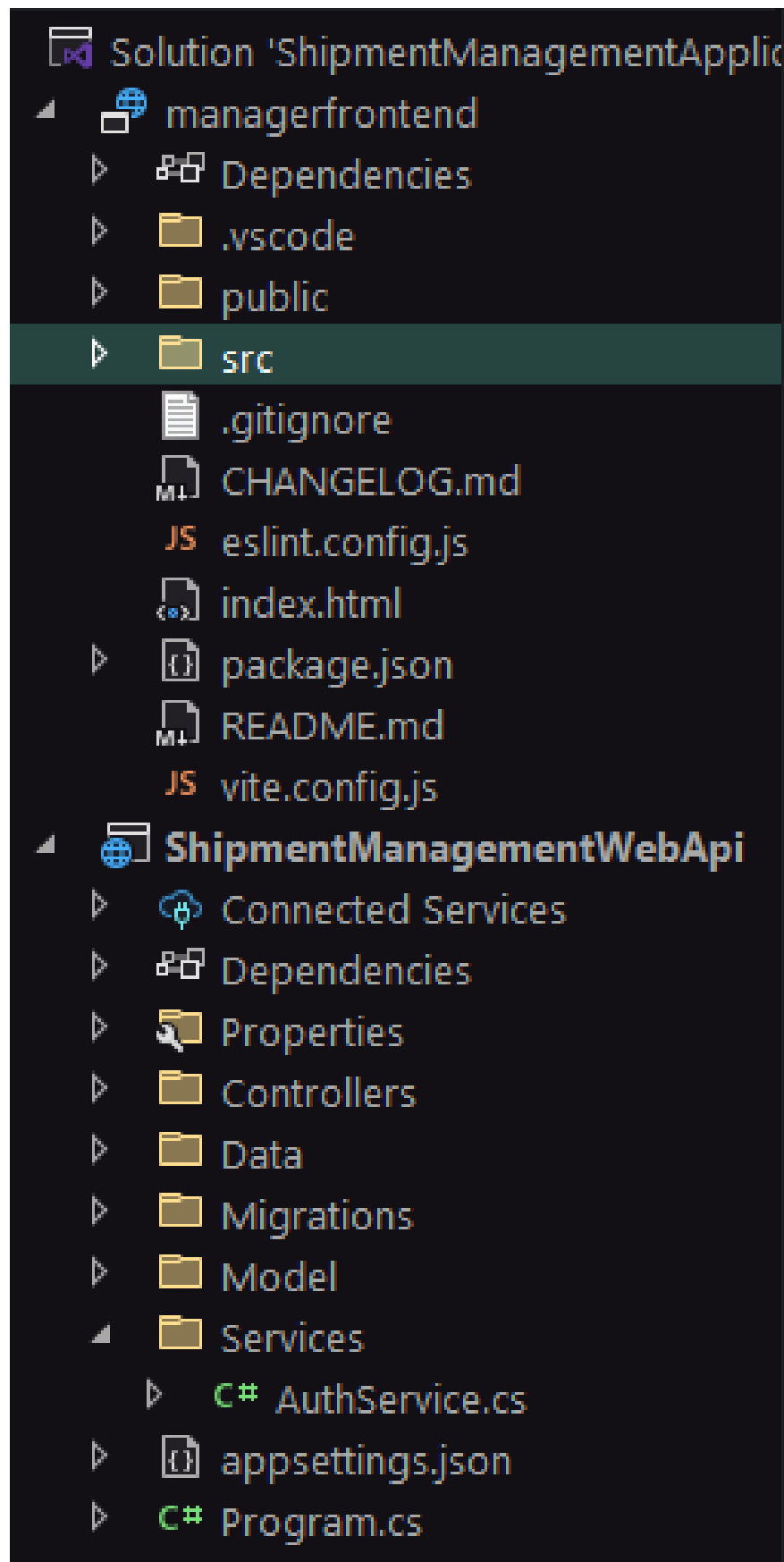
td

- Name: Cards, Type: Toys, Addressed To: Nothern Uganda.

Edit

Delete

Code Base



Backend:

/Services/AuthService.cs

```
using ShipmentManagementWebApi.Data;
using ShipmentManagementWebApi.Models;
using Microsoft.EntityFrameworkCore;

namespace ShipmentManagementWebApi.Services;

public class AuthService
{
    private readonly AppDbContext _context;

    public AuthService(AppDbContext context)
    {
        _context = context;
    }

    public User Authenticate(string username, string password)
    {
        return _context.Users.FirstOrDefault(u => u.Username == username && u.Password
== password);
    }

    public bool Register(User user)
    {
        if (_context.Users.Any(u => u.Username == user.Username)) return false;
        _context.Users.Add(user);
        _context.SaveChanges();
        return true;
    }
}
```

appsettings.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "ConnectionStrings": {
    "DefaultConnection":
"Server=127.0.0.1;Port=3306;Database=ShipmentDb;User=root;Password=root"
  },
  "AllowedHosts": "*"
}
```

Program.cs

```
using Microsoft.EntityFrameworkCore;
using ShipmentManagementWebApi.Data;
using ShipmentManagementWebApi.Services;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.

builder.Services.AddControllers();
// Learn more about configuring Swagger/OpenAPI at
https://aka.ms/aspnetcore/swashbuckle
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

// Configure EF Core with MySQL
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseMySQL(
        builder.Configuration.GetConnectionString("DefaultConnection"),

ServerVersion.AutoDetect(builder.Configuration.GetConnectionString("DefaultConnection"
))
));

// Register custom services
builder.Services.AddScoped<AuthService>();

builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowReactApp",
        policy => policy
            .WithOrigins("http://localhost:51097") // React dev server
            .AllowAnyHeader()
            .AllowAnyMethod());
});

var app = builder.Build();

app.UseCors("AllowReactApp");

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();

app.UseAuthorization();

app.MapControllers();

app.Run();
```


/Models/User.cs

```
using System.ComponentModel.DataAnnotations;

namespace ShipmentManagementWebApi.Models
{
    public class User
    {
        [Key]
        public int Id { get; set; }
        public string? Username { get; set; }
        public string? Password { get; set; } // Plain text for simplicity
        public string? Role { get; set; } = "User"; // Default role
    }
}
```

/Models/Shipment.cs

```
using System.ComponentModel.DataAnnotations;

namespace ShipmentManagementWebApi.Models
{
    public class Shipment
    {
        [Key]
        public int Id { get; set; }
        public string? Name { get; set; }
        public string? Type { get; set; }
        public string? Address { get; set; }
    }
}
```

/Data/AppDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using System.Collections.Generic;
using ShipmentManagementWebApi.Models;

namespace ShipmentManagementWebApi.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
        { }

        public DbSet<User> Users { get; set; }
        public DbSet<Shipment> Shipments { get; set; }
    }
}
```

/Controllers/AuthController.cs

```
using Microsoft.AspNetCore.Mvc;
using ShipmentManagementWebApi.Services;
using ShipmentManagementWebApi.Models;

[ApiController]
[Route("api/[controller]")]
public class AuthController : ControllerBase
{
    private readonly AuthService _authService;

    public AuthController(AuthService authService)
    {
        _authService = authService;
    }

    [HttpPost("register")]
    public IActionResult Register(User user)
    {
        user.Role = "User"; // Default role
        if (_authService.Register(user)) return Ok("Registered successfully");
        return BadRequest("Username already exists");
    }

    [HttpPost("login")]
    public IActionResult Login(User user)
    {
        var loggedInUser = _authService.Authenticate(user.Username, user.Password);
        if (loggedInUser == null) return Unauthorized("Invalid credentials");
        return Ok(loggedInUser); // Return user object with role
    }
}
```

/Controllers/ShipmentController.cs

```
using Microsoft.AspNetCore.Mvc;
using ShipmentManagementWebApi.Data;
using ShipmentManagementWebApi.Models;

[ApiController]
[Route("api/[controller]")]
public class ShipmentController : ControllerBase
{
    private readonly ApplicationDbContext _context;

    public ShipmentController(ApplicationDbContext context)
    {
        _context = context;
    }

    [HttpGet]
    public IActionResult GetAll() => Ok(_context.Shipments.ToList());

    [HttpGet("{id}")]
    public IActionResult Get(int id)
    {
        var shipment = _context.Shipments.Find(id);
        if (shipment == null) return NotFound();
        return Ok(shipment);
    }

    [HttpPost]
    public IActionResult Create(Shipment shipment, [FromQuery] string role)
    {
        if (role != "Admin") return Unauthorized("Only admin can create shipments");
        _context.Shipments.Add(shipment);
        _context.SaveChanges();
        return Ok(shipment);
    }

    [HttpPut("{id}")]
    public IActionResult Update(int id, Shipment updatedshipment, [FromQuery] string
role)
    {
        if (role != "Admin") return Unauthorized("Only admin can update shipments");
        var shipment = _context.Shipments.Find(id);
        if (shipment == null) return NotFound();

        shipment.Name = updatedshipment.Name;
        shipment.Type = updatedshipment.Type;
        shipment.Address = updatedshipment.Address;

        _context.SaveChanges();
        return Ok(shipment);
    }

    [HttpDelete("{id}")]
    public IActionResult Delete(int id, [FromQuery] string role)
    {
        if (role != "Admin") return Unauthorized("Only admin can delete shipments");
        var shipment = _context.Shipments.Find(id);
        if (shipment == null) return NotFound();

        _context.Shipments.Remove(shipment);
        _context.SaveChanges();
    }
}
```

```
        return Ok("Deleted");  
    }  
}
```

Frontend – React JS

/src/main.jsx

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';
import { AuthProvider } from './context/AuthContext';
import './index.css';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <AuthProvider>
    <App />
  </AuthProvider>
);
```

/src/App.jsx

```
import React, { useContext } from 'react';
import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';
import Login from './components/Login';
import Register from './components/Register';
import AdminDashboard from './components/AdminDashboard';
import UserDashboard from './components/UserDashboard';
import { AuthContext } from './context/AuthContext';
import './App.css';

function App() {
  const { user } = useContext(AuthContext);

  return (
    <Router>
      <Routes>
        <Route path="/" element={<Login />} />
        <Route path="/register" element={<Register />} />
        <Route path="/admin" element={user?.role === 'Admin' ? <AdminDashboard
/> : <Navigate to="/" />} />
        <Route path="/user" element={user?.role === 'User' ? <UserDashboard />
: <Navigate to="/" />} />
      </Routes>
    </Router>
  );
}

export default App;
```


/services/api.js

```
//api.js
import axios from 'axios';

const API = axios.create({
  baseURL: 'http://localhost:5099/api', // Adjust if needed
});

export default API;
```

/context/AuthContext.jsx

```
//AuthContext.jsx
import React, { createContext, useState } from 'react';

// eslint-disable-next-line react-refresh/only-export-components
export const AuthContext = createContext();

export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null); // Holds logged-in user

  return (
    <AuthContext.Provider value={{ user, setUser }}>
      {children}
    </AuthContext.Provider>
  );
};
```

/components/Login.jsx

```
import React, { useState, useContext, useEffect } from 'react';
import { useNavigate } from 'react-router-dom';
import API from '../services/api';
import { AuthContext } from '../context/AuthContext';

function Login() {
  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');
  const { setUser } = useContext(AuthContext);
  const { user } = useContext(AuthContext);
  const navigate = useNavigate();

  const handleLogin = async () => {
    if (username == '' || password == '') {
      alert("All credentials are mandatory.");
      return;
    }

    try {
      const res = await API.post('/auth/login', { username, password });
      setUser(res.data);
      res.data.role === 'Admin' ? navigate('/admin') : navigate('/user');
    } catch {
      alert('Invalid credentials');
    }
  };

  const handleRegister = () => {
    navigate('/register');
  };

  useEffect(() => {
    document.title = "Login";

    if (user?.role == "Admin") {
      navigate("/admin");
    }
    else if (user?.role == "User") {
      navigate("/user")
    }
  });

  return (
    <>
      <div class="container">
        <h2>Login</h2>
        <table>
          <tr>
            <td>
              <div class="input-group input-group-lg mb-3">
                <input class="form-control" placeholder="Username"
onChange={e => setUsername(e.target.value)} />
              </div>
            </td>
          </tr>
          <tr>
            <td>
              <div class="input-group input-group-lg mb-3">
                <input class="form-control" placeholder="Password"
type="password" onChange={e => setPassword(e.target.value)} />
              </div>
            </td>
          </tr>
        </table>
      </div>
    </>
  );
}
```

```

        </div>
      </td>
    </tr>
    <tr>
      <td>
        <button onClick={handleLogin}>Login</button>
      </td>
    </tr>
    <br><br>
    <tr>
      <td>
        <button onClick={handleRegister}>New User?
Register.</button>
      </td>
    </tr>
  </table>
</div>
</>
);
}

export default Login;

```

/components/Register.jsx

```
import React, { useState, useEffect, useContext } from 'react';
import { useNavigate } from 'react-router-dom';
import API from '../services/api';
import { AuthContext } from '../context/AuthContext';

function Register() {
  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');
  const { user } = useContext(AuthContext);
  const navigate = useNavigate();

  const handleRegister = async () => {
    if (username == '' || password == '') {
      alert("All credentials are mandatory.");
      return;
    }

    try {
      await API.post('/auth/register', { username, password });
      alert('Registration successful');
      navigate("/");
    } catch {
      alert('Registration failed');
    }
  };

  const handleLogin = () => {
    navigate('/');
  };

  useEffect(() => {
    document.title = "Register";

    if (user?.role == "Admin") {
      navigate("/admin");
    }
    else if (user?.role == "User") {
      navigate("/user")
    }
  });

  return (
    <>
      <div class="container">
        <h2>Register</h2>
        <table>
          <tr>
            <td>
              <div class="input-group input-group-lg mb-3">
                <input class="form-control" placeholder="Username"
onChange={e => setUsername(e.target.value)} />
              </div>
            </td>
          </tr>
          <tr>
            <td>
              <div class="input-group input-group-lg mb-3">
                <input class="form-control" placeholder="Password"
type="password" onChange={e => setPassword(e.target.value)} />
              </div>
            </td>
          </tr>
        </table>
      </div>
    </>
  );
}
```

```
        </td>
      </tr>
      <tr>
        <td>
          <button onClick={handleRegister}>Register</button>
        </td>
      </tr>
      <br><br>
      <tr>
        <td>
          <button onClick={handleLogin}>Old User? Login.</button>
        </td>
      </tr>
    </table>
  </div>
</>
  );
}

export default Register;
```

/components/AdminDashboard.jsx

```
import React, { useContext, useEffect, useState } from 'react';
import API from '../services/api';
import { AuthContext } from '../context/AuthContext';

function AdminDashboard() {
  const { user } = useContext(AuthContext);
  const [shipments, setShipment] = useState([]);
  const { setUser } = useContext(AuthContext);
  const [searchTerm, setSearchTerm] = useState('');
  const [form, setForm] = useState({ name: '', type: '', address: '' });
  const [editingId, setEditingId] = useState(null);

  const fetchShipments = async () => {
    const res = await API.get('/shipment');
    setShipment(res.data);
  };

  const createOrUpdateShipment = async () => {
    if (form.name == '' || form.type == '' || form.address == '') {
      alert("All fields Needed!");
      return;
    }

    if (editingId) {
      await API.put(`/shipment/${editingId}?role=${user.role}`, form);
      setEditingId(null);
    } else {
      await API.post(`/shipment?role=${user.role}`, form);
    }
    setForm({ name: '', type: '', address: '' });
    fetchShipments();
  };

  const deleteShipment = async (id) => {
    await API.delete(`/shipment/${id}?role=${user.role}`);
    fetchShipments();
  };

  const startEdit = (shipment) => {
    setForm(shipment);
    setEditingId(shipment.id);
  };

  const filteredShipments = shipments.filter(s =>
    s.name.toLowerCase().includes(searchTerm.toLowerCase()) ||
    s.type.toLowerCase().includes(searchTerm.toLowerCase()) ||
    s.address.toString().includes(searchTerm)
  );

  const logout = () => {
    setUser(null);
  };

  useEffect(() => {
    document.title = "Admin Dash";

    fetchShipments();
  }, []);
}
```

```
return (  
    <div class="container">  
        <h2>Admin Dashboard</h2>  
        <br></br>  
        <button onClick={logout} style={{ marginBottom: '1rem' }}>  
            Logout  
        </button>  
        <br></br>  
        <br></br>  
        <table>  
            <tr>  
                <td>  
                    <div class="input-group input-group-lg mb-3">  
                        <input class="form-control" placeholder="Name"  
value={form.name} onChange={e => setForm({ ...form, name: e.target.value })} />  
                    </div>  
                </td>  
            </tr>  
            <tr>  
                <td>  
                    <div class="input-group input-group-lg mb-3">  
                        <input class="form-control" placeholder="Type"  
value={form.type} onChange={e => setForm({ ...form, type: e.target.value })} />  
                    </div>  
                </td>  
            </tr>  
            <tr>  
                <td>  
                    <div class="input-group input-group-lg mb-3">  
                        <input class="form-control" placeholder="Address"  
value={form.address} onChange={e => setForm({ ...form, address: e.target.value })} />  
                    </div>  
                </td>  
            </tr>  
            <tr>  
                <td>  
                    <button onClick={createOrUpdateShipment}>  
                        {editingId ? 'Update Shipment' : 'Add Shipment'}  
                    </button>  
                </td>  
            </tr>  
            <br></br>  
            <br></br>  
            <br></br>  
            <tr>  
                <td>  
                    <input  
                        class="form-control"  
placeholder="Search by name, type, or address"  
value={searchTerm}  
onChange={e => setSearchTerm(e.target.value)}  
                    />  
                </td>  
            </tr>  
        </table>  
        <ul>  
            {filteredShipments.map(s => (  
                <li key={s.id}>  
                    <strong>Name:</strong> {s.name},&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&~  
                    <strong>Type:</strong> {s.type},&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&~  
To:</strong> {s.address} &nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&~  
                    <button onClick={() => startEdit(s)}>Edit</button>  
~  
                    <button onClick={() => deleteShipment(s.id)}>Delete</button>  
                </li>  
            )}
```



```
    }}  
    </ul>  
  </div>  
);  
}  
  
export default AdminDashboard;
```

/components/UserDashboard.jsx

```
import React, { useContext, useEffect, useState } from 'react';
import API from '../services/api';
import { AuthContext } from '../context/AuthContext';

function UserDashboard() {
  const [shipments, setShipment] = useState([]);
  const [searchTerm, setSearchTerm] = useState('');
  const { setUser } = useContext(AuthContext);

  useEffect(() => {
    document.title = "User Dash";

    API.get('/shipment').then(res => setShipment(res.data));
  }, []);

  const logout = () => {
    setUser(null);
  };

  const filteredShipments = shipments.filter(s =>
    s.name.toLowerCase().includes(searchTerm.toLowerCase()) ||
    s.type.toLowerCase().includes(searchTerm.toLowerCase()) ||
    s.address.toString().includes(searchTerm)
  );

  return (
    <div class="container">
      <h2>User Dashboard</h2>
      <br></br>
      <button onClick={logout} style={{ marginBottom: '1rem' }}>
        Logout
      </button>
      <br></br>
      <br></br>
      <table>
        <tr>
          <td>
            <input
              class="form-control"
              placeholder="Search by name, type, or address"
              value={searchTerm}
              onChange={e => setSearchTerm(e.target.value)}
            />
          </td>
        </tr>
      </table>
      <br></br>
      <br></br>
      <ul>
        {filteredShipments.map(s => (
          <li key={s.id}>
            <strong>Name:</strong> {s.name},&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&
            <strong>Type:</strong> {s.type},&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&   <strong>Addressed To:</strong> {s.address} &nbsp;&nbsp;&nbsp;&nbsp;&~
          </li>
        ))}
      </ul>
    </div>
  );
}
```

```
export default UserDashboard;
```